VHDL Coding Samples

Note: Entity is not included; SIGNAL types could be deducted from the codes or clarified as comments.

I. COMBINATIONAL CIRCUITS

1A. 2_to_4 DECODER with active low enable and active low outputs : use WHEN … ELSE construct

```
ARCHITECTURE my_arch OF decoder_2_to_4_en IS
BEGIN
    y <= "1111" WHEN (en = '1')      ELSE          --decoder outputs disabled
         "1110" WHEN (sel = "00")  ELSE
         "1101" WHEN (sel = "01")  ELSE
         "1011" WHEN (sel = "10")  ELSE
         "0111";                   -- otherwise
END my_arch ;
```

1B. 2_to_4 DECODER with active low enable and active low outputs : Use WITH SELECT construct

```
ARCHITECTURE my_arch OF decoder_2_to_4_en IS
SIGNAL  en_sel_aggregate : STD_LOGIC_VECTOR (2 DOWNTO 0);
BEGIN
    en_sel_aggregate <=   ( en, sel(1), sel(0) );      -- could also use the concatenate operator &:
                                                       --  en_sel_aggregate <= en & sel;
    WITH ( en_sel_aggregate ) SELECT
    y <= "1111" WHEN "100"|"101"|"110"|"111",   -- | is ORing conditions; decoder outputs disabled
         "1110" WHEN "000",
         "1101" WHEN "001",
         "1011" WHEN "010",
         "0111" WHEN others;                           --"011" or other combinations
END my_arch ;
```

1C. 2_to_4 DECODER with active low enable and active low outputs : Use PROCESS, IF THEN constructs

```
ARCHITECTURE my_arch OF decoder_2_to_4_en IS
BEGIN
Decoder: PROCESS (en, sel)
    BEGIN
        IF (en = '1') THEN                    -- decoder outputs disabled
          y <= "1111";
        ELSIF (sel="00") THEN                 –- select y0
            y <= "1110";
            ELSIF (sel="01") THEN             –- select y1
                y <= "1101";
                ELSIF (sel="10") THEN         –- select y2
                    y <= "1011";
                ELSE                          -- combinational : all possible combinations covered
                    y <= "0111";              –- select y3
        END IF;
    END PROCESS;
END my_arch ;
```

1D. 2_to_4 DECODER with active low enable and active low outputs : Use PROCESS, CASE constructs

```
ARCHITECTURE my_arch OF decoder_2_to_4_en IS
BEGIN
    Decoder: PROCESS (en, sel)
        VARIABLE  en_sel_concatenate : STD_LOGIC_VECTOR (2 DOWNTO 0);
    BEGIN
        en_sel_concatenate := en & sel;
        CASE en_sel_concatenate IS
            WHEN "100"|"101"|"110"|"111"=>
                y <= "1111";                    -- decoder outputs disabled
            WHEN "000" =>
                y <= "1110";
            WHEN "001" =>
                y <= "1101";
            WHEN "010" =>
                y <= "1011";
            WHEN others =>
                y <= "0111";
        END CASE;
     END PROCESS;
END my_arch ;
```

2. 4-bit Magnitude Comparator with agtb (greater than), altb (less than), and aeqb (equal) outputs

```
ARCHITECTURE my_arch OF Mag_comparator4 IS
BEGIN
      aeqb <= '1' WHEN a = b ELSE                    -- aeqb is BUFFER type output
                ‘0’;
      agtb <= a(3) WHEN ( a(3) /= b(3) ) ELSE        -- agtb is BUFFER type output
               a(2) WHEN ( a(2) /= b(2) ) ELSE
               a(1) WHEN ( a(1) /= b(1) ) ELSE
               a(0) WHEN ( a(0) /= b(0) ) ELSE
               '0';
       altb <= '1' WHEN  aeqb = ‘0’ AND agtb = ‘0’  ELSE
               '0';
END my_arch ;
```

3. 4-bit Barrel Shifter

```
-- shift_ctrl is input of type STD_LOGIC_VECTOR( 1 DOWNTO 0)
-- x are inputs of type STD_LOGIC_VECTOR(3 DOWNTO 0)
-- y are outputs of type STD_LOGIC_VECTOR(3 DOWNTO 0)
ARCHITECTURE my_arch OF Barrel_Shifter4 IS
BEGIN
      WITH  shift_ctrl  SELECT
        y <=  x                                  WHEN "00",     -- no rotate ( stay the same)
              x(2 DOWNTO 0) & x(3)               WHEN "01",              -- rotate left 1 bit
              x(1 DOWNTO 0) & x(3 DOWNTO 2)      WHEN "01",              -- rotate left 2 bit
              x(0) & x(3 DOWNTO 1)               WHEN OTHERS;            -- rotate left 3 bit
END my_arch ;
```

II. SEQUENTIAL CIRCUITS

4A. J-K FLIPFLOP, use WHEN .. ELSE and PROCESS. Coding style: two segments

```
ARCHITECTURE two_segment_arch OF JK_flipflop IS
SIGNAL present_q,  next_q : STD_LOGIC;
BEGIN                   -- j, k, clk are inputs of type STD_LOGIC; q is output of type STD_LOGIC
-- to determine next_q (combinational )
     next_q <=  present_q   WHEN (  j = '0'  AND  k = '0')  ELSE      -- q unchanged (stay the same)
                '0'         WHEN (  j = '0'  AND  k = '1')  ELSE      -- q cleared
                '1'         WHEN (  j = '1'  AND  k = '0')  ELSE      -- q set
                NOT (present_q);                                      -- J K = “11”   q toggled
     q <= present_q;

-- Flip Flop ( D-type)
     Sequential: PROCESS(clk)
     BEGIN
          IF (clk’EVENT AND clk = '1') THEN
                present_q <= next_q;
          END IF;
     END PROCESS;
END two_segment _arch ;
```

4B. J-K FLIPFLOP, use PROCESS and IF THEN; Coding style: one segment

```
ARCHITECTURE one_segment_arch OF ex4B IS
     SIGNAL next_q : STD_LOGIC;
BEGIN                   -- j, k, clk are inputs of type STD_LOGIC; q is output of type STD_LOGIC
    one_segment: PROCESS(clk, j, k)
       VARIABLE  jk : STD_LOGIC_VECTOR (1 DOWNTO 0);
    BEGIN
        Jk :=  j & k;
        IF (clk'EVENT AND clk = '1') THEN
             IF (jk = "01") THEN                        -- clear Q
                  next_q <= '0';
             ELSIF (jk = "10")THEN                      -- set Q
                       next_q <= '1';
                  ELSIF (jk = "11")THEN                 -- Toggle Q
                            next_q <= NOT(next_q);
             END IF;
        END IF;
    END PROCESS;
    q <= next_q;                                        -- to output
END one_segment_arch ;
```

5A. 8-bit Counter with synchronous clear (higher priority) and synchronous load. Coding style: one segment

```
ARCHITECTURE one_segment_arch OF cnt8A IS
BEGIN            -- clrn, loadn, clk are inputs of type STD_LOGIC; q is output of type integer range 0 to 255
    PROCESS (clk)
        VARIABLE     cnt       : INTEGER RANGE 0 TO 255;
    BEGIN
        IF (clk'EVENT AND clk = '1') THEN
            IF (clrn = '0') THEN
                cnt := 0;                          -- synchronous clear ( higher priority)
            ELSIF (loadn = '0') THEN
                    cnt := d;                      -- synchronous load
                ELSE
                    cnt := cnt + 1;                -- count on each positive clock edge
            END IF;
        END IF;
        q  <=  cnt;                                -- to output
    END PROCESS;
END one_segment_arch;
```

5B. 8-bit Counter with synchronous clear (higher priority) and synchronous load. Coding style: two segments

```
ARCHITECTURE two_segment_arch OF cnt8B IS
   SIGNAL present_cnt, next_cnt : INTEGER RANGE 0 TO 255;
BEGIN            -- clrn, loadn, clk are inputs of type STD_LOGIC; q is output of type integer range 0 to 255
     PROCESS (clk)
     BEGIN
        IF (clk'EVENT AND clk = '1') THEN     -- registered
            present_cnt  <=  next_cnt;
        END IF;
     END PROCESS;
     next_cnt  <=  0   WHEN clrn = '0'    ELSE  -- combinational logic
                   d    WHEN loadn = '0'  ELSE
                   present_cnt  +  1;
      q <= present_cnt;                       -- to output
END two_segment_arch;
```

6A. Binary counter with max_count output: when the counter reaches 15, max_count output goes high for one cycle

```
PROCESS (clk, cnt)          -- clk is input of type STD_LOGIC, max_cnt is output of type STD_LOGIC
BEGIN                       -- q is output of type integer range 0 to 15
    IF (clk'EVENT AND clk = '1') THEN
        cnt <= cnt + 1;   -- cnt is a signal of  integer type range 0 to 15
    END IF;
    IF (cnt = 15) THEN
        max_cnt <= '1';                -- pulse max_cnt output
    ELSE
        max_cnt <= '0';                -- clear max_cnt
    END IF;
END PROCESS;
q<= cnt;                               -- to output
```

6B. Binary counter with incorrect max_count output at cnt = 0

```
Incorrect: PROCESS (clk)      -- clk is input of type STD_LOGIC, max_cnt is output of type STD_LOGIC
BEGIN                                          -- q is output of type integer range 0 to 15
    IF (clk'EVENT AND clk = '1') THEN
        cnt <= cnt + 1;                        -- cnt is a signal of  integer type range 0 to 15
        IF (cnt = 15) THEN
            max_cnt <= '1';  -- max_cnt is registered, i.e. it is the output of a D-Flipflop delayed one clock cycle
        ELSE                  -- instead pulse max_cnt output at max count 15, it is pulsed at cnt = 0
            max_cnt <= '0';   -- clear max_cnt
        END IF;
    END IF;
END PROCESS;
q<= cnt;                          -- to output
```

7A. PWM: period 16 cycles, duty cycle = 5/16. Coding style: one segment

```
ARCHITECTURE one_segment_arch OF pwm_oneseg IS
    SIGNAL cnt : INTEGER RANGE 0 TO 15;
BEGIN              -- clk is input of type STD_LOGIC; q is output of type STD_LOGIC;
    PROCESS (clk)                                  -- may want to include reset for cnt
    BEGIN
        IF (clk'EVENT AND clk = '1') THEN          -- registered
            cnt <= cnt + 1;
            IF (cnt < 5) THEN
                q <= '1';
            ELSE
                q <= '0';
            END IF;
        END IF;
    END PROCESS;
END one_segment_arch;
```

7B. PWM: period 16 cycles, duty cycle = 5/16. Coding style: two segments

```
ARCHITECTURE two_segment_arch OF pwm IS
    SIGNAL present_cnt, next_cnt : INTEGER RANGE 0 TO 15;
    SIGNAL present_pwm, next_pwm : STD_LOGIC;
BEGIN              -- clk is input of type STD_LOGIC; q is output of type STD_LOGIC;
    PROCESS (clk)                                  -- may want to include reset for cnt
    BEGIN
        IF (clk'EVENT AND clk = '1') THEN          -- registered
            present_cnt  <=  next_cnt;
            present_pwm <= next_pwm;
        END IF;
    END PROCESS;
    next_cnt  <=  present_cnt + 1;                 -- combinational logic
    next_pwm <= '1'    WHEN present_cnt  < 5  ELSE -- duty cycle = 5/16
                '0';
    q <= present_pwm;                  -- to output
END two_segment_arch;
```

Useful VHDL Codes

8. To detect edges of a signal named mclock

```
-- Assume signals edge_detected,  mclock_rising, and mclock_falling are all '0' initially after reset
-- Codes not shown here.

IF (clk50'EVENT AND clk50 = '1') THEN
    IF ( mclock = '1') THEN
        IF (edge_detected = '0' ) THEN          -- edge_detected is the last mclock logic value
            mclock_rising <= '1' ;              -- mclock_rising is a flag to indicate a positive edge
                                                --     is detected; active for one cycle
        ELSE
            mclock_rising <= '0' ;
        END IF;
        edge_detected := '1';                   -- to cause mclock_rising active for one cycle
    ELSE
        IF (edge_detected = '1' ) THEN          -- edge_detected is the last mclock logic value
            mclock_falling <= '1' ;             -- mclock_falling is a flag to indicate a falling edge
                                                --    is detected; active for one cycle
        ELSE
            mclock_falling <= '0' ;
        END IF;
        edge_detected := '0';                   -- to cause mclock_falling active for one cycle
    END IF;
END IF;
```

9. Handshake between a device and a host.
Before the host reads the device data, the host sends a read-request ReadReq, and then waits for data ready signal DataReady to be active. When DataReady is active, device data is ready to be picked up.

```
Handshake: PROCESS( ReadReq, ReadyFlag)
BEGIN
    IF ( ReadReq = '1') THEN        --ReadReq is set by the host host
            DataReady <= '0';       -- not ready
    ELSIF (ReadyFlag'EVENT AND ReadyFlag = '1') THEN
            DataReady <= '1';       -- ready.  ReadyFlag is set by the device when data is ready
    END IF;
END PROCESS;
```

10. To use a signal to trigger an event at a specific time

```
IF (clk50'EVENT  AND clk50 = '1' ) THEN
    count := count + 1;
    IF (count = 5000) THEN
        timeup <= '1';          -- timeup is active once every 100 µsec (50 MHz clock)
    ELSE
        timeup <= '0';
        count := 0;
    END IF;
END IF;
```

11. To stay in the same state for ten clock cycles
--In this example, Somestate is used to shift data_in into data ten times (ten clock cycles)

```
IF ( clk50'EVENT  AND clk50 = '1' ) THEN
    CASE state IS
        WHEN SomeState =>       -- receive ten bits of  data
            data  <= data(9 DOWNTO 1) & data_in;  -- shift data_in on each clock cycle
            cnt := cnt + 1;             -- cnt = 0 upon reset
            IF(cnt = 10) THEN
                state <= NextState;
                cnt := 0;
            ELSE
                state <= SomeState;
            END IF;
        . . . . .
```

12. To stay in the same state forever. To be used when a sequential circuit has completed its task and will no longer needed any more.

```
IF ( clk50'EVENT  AND clk50 = '1' ) THEN
    CASE state IS
        WHEN IDLE =>
            state <= IDLE;
```